Governed Setpoint Writing for Industrial Agents: A Configuration-Driven Runtime and Simulated Evaluation

***Abstract*—Industrial agents can propose setpoint changes, but executing a proposal requires preserving production context, signal meaning, write authority, and process limits. We present a configuration-driven runtime that binds production context and named signals to operations and registered protocol drivers, and routes agent-proposed and direct calls through a shared dispatcher. Write handlers resolve live context, apply authorization and configured limits, and pass approved requests through a common guard before driver dispatch. Command receipts, actuator readbacks, and later process observations are recorded separately. In benchmark-mode simulated production scenes, probes checked selected node-to-driver mappings: all 220 admissible requests were accepted, 219 collected structural-range requests were rejected, and 216 available readbacks matched. A scripted mission reached its target band in all 15 seed–scene runs, whereas a 60-s no-write condition did not; because exposure time and mission logic differed, this is a descriptive demonstration, not a control-performance comparison. Removing the process-window guard allowed all six targeted violations to reach a mock driver, while structural checks remained active. A live-agent pilot issued no governed writes. The results support the tested configuration and guard paths in simulation; they do not establish live-agent performance, human-approval effects, or physical-plant safety.**

***Index Terms*—Industrial agents, governed setpoint writing, industrial communication, simulation-based evaluation, system integration.**

I. Introduction

INDUSTRIAL agents can propose parameter changes, but a proposal becomes an industrial operation only after the system resolves its production context and signal meaning. Each write must be tied to the active line, product, recipe, and run; mapped to a configured control point; and checked against current authority and process limits. The process measurement used to assess a response is distinct from the command signal. In the cast-film scene studied here, line speed is the writable setpoint and film thickness is measured separately, although both use Message Queuing Telemetry Transport (MQTT). A command receipt or setpoint readback therefore does not establish the film response. Fig. 1 illustrates the distinction.

Research on industrial agents addresses service supervision, control generation, programmable logic controller (PLC) programming, and multi-agent manufacturing workflows [1]–[10]. Other work exposes manufacturing capabilities through standardized tools and semantic skills [11]–[13], or combines agent proposals with asset models, approval, audit, and process verification [14]–[16]. Together, these studies address agent organization, plant-facing tools, and operation governance. The design problem is to carry the active line–product–recipe–run context and signal meaning through authorization and protocol dispatch, then return evidence that separates the command outcome from the process response.

We implement this path as a configuration-driven runtime. A production-scene description binds the line–product–recipe–run context to semantic acquisition and control nodes, registered protocol drivers, process objectives, and write limits. Task-mediated and direct operation calls enter a shared dispatcher, where route handlers resolve live state again before device access. Service-owned records keep the requested command, optional device readback, and later process observation distinct. Through a registered harness, the Agent decides whether to request another task-level step; the runtime applies the configured checks to each operation. The paper contributes an implemented system-integration architecture; it does not propose a new control law or agent-reasoning algorithm.

The contributions are:

1) A production-scene configuration model that binds operational context to semantic signal acquisition and control, registered protocol drivers, process objectives, and engineering limits behind named operations.
2) A task-to-device path that exposes the same named operations through registered Agent harness adapters and direct callers. Both paths use shared dispatch checks; route-specific handlers apply current admission and approval rules and retain command evidence separately from process observations. The harness path is specified and its orchestration is exercised, but the positive blocks use direct or scripted clients; the live-Agent pilot is reported as a negative result.
3) An evaluation of selected scene–protocol mappings, a scripted target-band demonstration, a process-window guard ablation, route checks, and a negative live-Agent pilot. Each block uses its own unit and denominator; the study reports descriptive counts and does not claim population-level inference.

Section II reviews industrial-agent and plant-integration work; Section III presents the architecture and scene configuration. Section IV traces a task-level operation through its feedback path, and Section V specifies write governance and evidence. Section VI reports the simulator and mock-driver evaluation. Section VII discusses what the results support and what remains untested.

II. Related Work

Work relevant to this system spans industrial-agent supervision, agent-facing manufacturing interfaces, and operation governance. Supplementary Table S7-I compares representative systems; it does not rank their performance.

A. Industrial Agents and Plant Integration

Industrial agents have been used to supervise production services, generate control actions, program programmable logic controllers (PLCs), and coordinate manufacturing tasks. Biskupovic et al. study online service selection and reselection in service-oriented industrial cyber–physical systems [1]. Xia et al. connect language-model agents, digital twins, and automation systems [17]; other work examines agent-generated control and PLC programs [2], [5], [6]. Recent multi-agent studies evaluate module-level agents and skill execution in simulated manufacturing [4], [8]. These studies focus primarily on agent organization, control generation, or production coordination.

Agent-facing interfaces offer a second point of comparison. Vieira da Silva et al. examine MCP for exposing manufacturing capabilities as tools, and Köhle et al. compose semantic recipes from OPC UA skills [11], [12]. Xavier et al. present IndustriConnect, an MCP adapter set for Modbus, MQTT/Sparkplug B, and OPC UA, tested with a mock-first workflow [13]. Liu et al. use agents and MCP to commission a cognitive digital twin by extracting specifications and binding live telemetry [10]. Köhle et al. combine module-specific agents, OPC UA skills, and MQTT coordination in nine simulated production challenges [4]. Together, these studies address tool exposure, protocol connectivity, semantic commissioning, and multi-agent execution. This paper focuses on the operation boundary that carries an active production tuple and named signal mappings through route-specific governance to a configured driver, while keeping command and process evidence distinct. The evaluation tests these paths; it is not a comparative performance study.

B. Governed Agent Operations

Safety and audit mechanisms have been integrated with agent execution in several forms. Graul et al. ground proposals in the Asset Administration Shell (AAS) and use signed approval, PLC-validated OPC UA Methods, and audit logging [14]. Schall separates large language model (LLM) proposals from verification grounded in piping and instrumentation diagrams (P&IDs), supervisory control and data acquisition (SCADA), and standard operating procedures (SOPs), while Lin and Chen report a policy-governed sensor–manufacturing execution system (MES)–enterprise resource planning (ERP) digital thread [15], [16]. Runtime enforcement and capability specification have also been proposed for agent tool use [18], [19], while industrial security and safety standards define separate zones, conduits, and safety-instrumented functions [20], [21]. The runtime studied here applies admission and approval between a named operation and its configured driver. PLCs, safety instrumented systems (SIS), and other plant-protection functions remain separate safeguards.

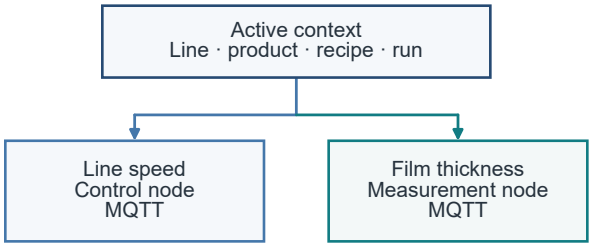


Fig. 1. In the cast-film scene, the line-speed setpoint and film-thickness measurement are separate configured nodes even though both use MQTT. A command receipt or optional setpoint readback is distinct from the later process observation.

C. Industrial Context and Operation Semantics

MCP standardizes tool discovery and invocation, while operation-level scope, policy, grant, and resource-binding checks remain application concerns [22]. ISA-88 and ISA-95 define recipe and production-operation concepts [23], [24]; run-to-run control relates process observations to later decisions [25]; human–automation research analyzes responsibility allocation [26]. These define the production entities and responsibilities the runtime must preserve. Section III explains how they are carried into configured operations; Section VI tests selected routes in simulation and against a mock driver.

III. System Design

The runtime exposes named services through two entry paths. Task-mediated calls pass through the collaboration runtime and a registered Agent harness. Direct calls through the Representational State Transfer (REST) API or the standard input/output (stdio) transport for the Model Context Protocol (MCP) enter the shared dispatcher without creating a task turn. Both paths use the same scene configuration for production context, semantic-node mappings, protocol endpoints, objectives, and write limits.

A. Design Rationale and Requirements

The runtime must preserve process meaning across heterogeneous drivers, provide a stable operation contract for registered harness adapters, and check current authority and process limits before each write. It must also distinguish command acceptance from later process observations. Scene configuration, registered adapters, shared dispatch, route handlers, and service-owned records enforce these boundaries.

At task start, the context builder assembles the active production tuple and relevant process summaries. This context informs a proposal but does not grant write authority; each operation resolves live state again at invocation.

B. Layered Architecture

Fig. 2 shows the system layers and their interfaces. Task and chat routes use the collaboration runtime, which provides workspaces, channels, task queues, shared memory, mailboxes, task templates, and harness adapters. An adapter connects an agent engine to its task interface; its binding and channel policy constrain the tools available to that agent.

Direct REST calls and the stdio-MCP operation bridge bypass task execution and enter the shared dispatcher. They create neither a task nor a harness turn; MCP management and task methods remain on their service routes. The scene

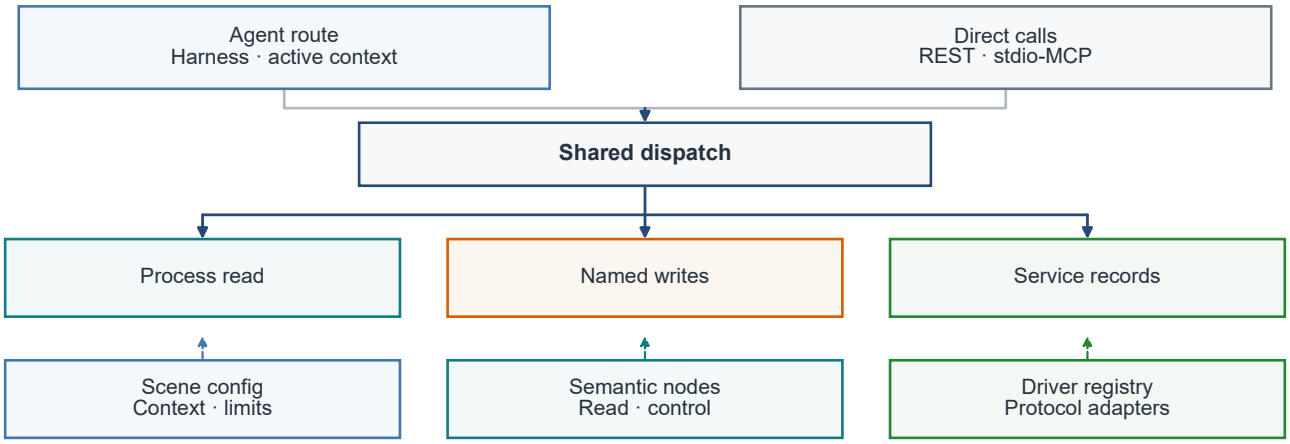


Fig. 2. The system routes task-mediated and direct operation calls through shared dispatch and service handlers. Solid arrows show operation flow and configured field mappings; dashed connectors associate the scene configuration with the services that consume it. Named write operations use the shared write controller, while service-owned records keep approvals, command receipts, optional readbacks, and later observations distinct.

configuration has two consumers: the task route receives active context and objectives, while operation services resolve node mappings and guards when an operation executes. The service band provides data acquisition (DAQ), write control, analysis, retrieval, and optional model services. At the field layer, input/output (I/O) mappings connect semantic nodes to built-in or plugin drivers for a plant or simulator. Each owning service retains its task, command, observation, or configuration records and associates them with the actor and production context where supported.

C. Scene Configuration as the Integration Boundary

Scene-specific process semantics are represented in configuration before an agent task begins. The active line identifies the production context; product and recipe records supply applicable targets and limits; the run record provides the current operating state. Semantic nodes bind process variables to direction-specific acquisition or control mappings, and the driver registry resolves each mapping to a supported protocol implementation. Here *semantic* denotes a named process variable with a direction, unit, and configured mapping; it is not an OPC UA or Asset Administration Shell information model. The agent-facing operation remains named and typed, so a harness does not need to construct protocol-specific messages. A scene using an already registered driver can supply different context and node mappings behind the same operation contract. A protocol not present in the registry still requires a driver adapter; configuration alone does not implement a new protocol.

Table I summarizes how the scene description is consumed along the execution path. It distinguishes configuration-time bindings from checks against live state. The context assembled for an agent turn is descriptive: before a write, the handler resolves the active context again and applies the current limits. Approval is bound to the proposal and to the context where that route supports context snapshots; the command receipt remains separate from subsequent DAQ observations.

D. Cross-Module Execution Flow

For a task-mediated call, the registered adapter presents a scoped tool catalog and current task context to the Agent. At the operation boundary, the shared dispatcher checks the predicates declared for the named operation before selecting its handler. The handler resolves live production state, applies route-specific guards and limits, and requests any required approval before device I/O.

For an industrial operation u by agent a in channel c, define the shared dispatch condition as

$$\mathcal{A}(a,c,u) = \bigwedge_{X \in \mathcal{C}(u)} X(a,c,u), \tag{1}$$

where S, P, G, and B denote invocation scope, current channel policy, applicable industrial grant, and the agent's binding to referenced resources, respectively. $\mathcal{C}(u) \subseteq \{S,P,G,B\}$ is the set of predicates declared for operation u; it always contains S and P. The grant predicate G is included when the resolved channel, target line, and owner/grant policy require it; explicit exemptions omit it. The binding predicate B is included when the operation declares resource references. A declared no-reference operation omits B. Each included check must pass before handler selection. The policy predicate includes channel tool selection, profile, and duty-separation checks where configured. After these predicates pass, the selected route applies its operational guards and any required approval. The checks performed after approval depend on the route; Section V specifies the single-node and recipe-operation paths.

For a direct REST call, the service establishes a and c before applying these checks. An agent token must match the requested agent identifier; an authenticated user must be the channel owner or an administrator. The manager loads the channel-agent record and rechecks scope and policy, followed by any applicable grant and binding checks. Handlers resolve live production state from request identifiers and owning services.

TABLE I
SCENE CONFIGURATION ELEMENTS AND THEIR ROLES IN THE RUNTIME PATH.

Element	Configured information	Runtime use
Production context	Line, product, recipe, and run associations	Context construction for a task turn; live context resolution before a write
Semantic I/O	Process variable, direction, unit, and node mapping	Acquisition uses the read mapping; accepted operations resolve the configured control mapping
Driver binding	Supported protocol driver and endpoint configuration	Driver registry selects the protocol implementation after operation checks
Operation admission	Engineering, baseline, product, and recipe limits; step and timing rules where configured	Route handler checks the candidate against current bounds and rejects inadmissible requests
Approval policy	Whether approval is required and the decision validity window	Pending request binds the proposed action; route-specific checks run after approval
Evidence ownership	Command receipt, verification grade, and later DAQ observations	Owning services retain command outcomes separately from process measurements

Section V specifies route-level admission, timing, approval, and evidence rules for both task-mediated and direct calls.

IV. AGENT-COORDINATED TASK-LEVEL OPERATION

A task supplies a process objective and the active production context, including line, product, and run. Fig. 3 presents the system-level task, agent-team, runtime, and governance view. Its *BINGO/REACH THE GOAL* panel denotes the application objective; it does not guarantee that one optimization call autonomously reaches the target. The task context alone does not authorize a device write. The control node is resolved from an explicit `control_node_id` or the Agent's DCW/recipe binding. The DAQ target is resolved from an explicit target ID or the Agent's DAQ binding; a read-only node visible through the channel line is also eligible.

In goal-mode execution, a Lead Agent may decompose the objective, dispatch subtasks to worker agents, and judge completion. This optional coordination structures task work but does not define a control policy. For each control step, the responsible Agent invokes `optimization_explore`; that operation performs the checks before device I/O.

The operation resolves the optimization profile and control-node binding, then uses the node's stored value as the current setpoint u_0. This value is not necessarily a fresh device readback. If direction is omitted, the operation defaults to an upward adjustment; otherwise, the supplied direction determines $s \in \{-1, +1\}$. For a step Δu, it forms

$$u^* = \mathrm{clip}(u_0 + s\Delta u,\, u_{\min},\, u_{\max}), \tag{2}$$

where configured tuning bounds take precedence; node bounds are used otherwise. The effective step is selected in this order: an explicit argument, the binding's tuning step, the scene's maximum step, or one twentieth of the node range.

The operation then resolves and verifies the DAQ target and its visibility, and runs the intervention-identifiability preflight, which is enabled by default but can be explicitly disabled. It also checks available model and ontology evidence: a confirmed conflict blocks dispatch, while missing evidence is treated as unknown. If these checks pass, the operation attempts to collect a pre-write DAQ window; its mean may be unavailable. Profile, binding, DAQ-visibility, identifiability, and blocking cross-check failures return before an exploration record is created.

A valid candidate enters the shared write-control path, which resolves the applicable binding and authority and determines whether the route requires approval. Only a gated route waits for approval. After approval, the path rechecks authorization and binding; the common controller then applies the effective range, step, timing, and hold constraints before calling the configured driver. A route without an approval gate skips the wait but still passes through the common controller checks. A write denial or approval decline is recorded as rejected without driver dispatch. A driver-stage error may also be recorded as rejected after dispatch has begun, so status alone cannot establish whether the device changed; the caller must read and reconcile device state before retrying.

After a successful write, the operation waits for the configured settling interval (10–300 s; 45 s by default) and attempts a post-write DAQ read. Let t_0 be the timestamp taken before the pre-write DAQ read and governed write call. The pre-write window is queried over $[t_0 - \tau, t_0]$, and the post-write window over $[t_0, t_1]$ after the settling wait τ. Because approval may delay dispatch, the latter window can include pre-dispatch samples; its mean alone is therefore not a causal estimate of the write's effect.

The exploration record retains the control value (the proposed target after a successful write), step, available DAQ means, and write status/message. For a rejected attempt, it retains the status/message but no separate attempted-target field. The shared write path separately records the command outcome and any device readback. On a later task turn, the Agent reviews the returned result and chooses another direction and step or stops. This is a discrete, Agent-mediated feedback process; the runtime does not iterate autonomously to convergence. Fig. 3 provides the broader system context for this operation; the ordered execution sequence is described above.

V. OPERATION GOVERNANCE AND EVIDENCE

Section III defines the shared dispatch contract. This section focuses on the checks that follow it: scene-specific write admission, conditional approval, and the records returned by each route.

A. Scene-Specific Write Admission

After the shared dispatch checks pass, the selected handler applies the limits configured for the target scene and control.

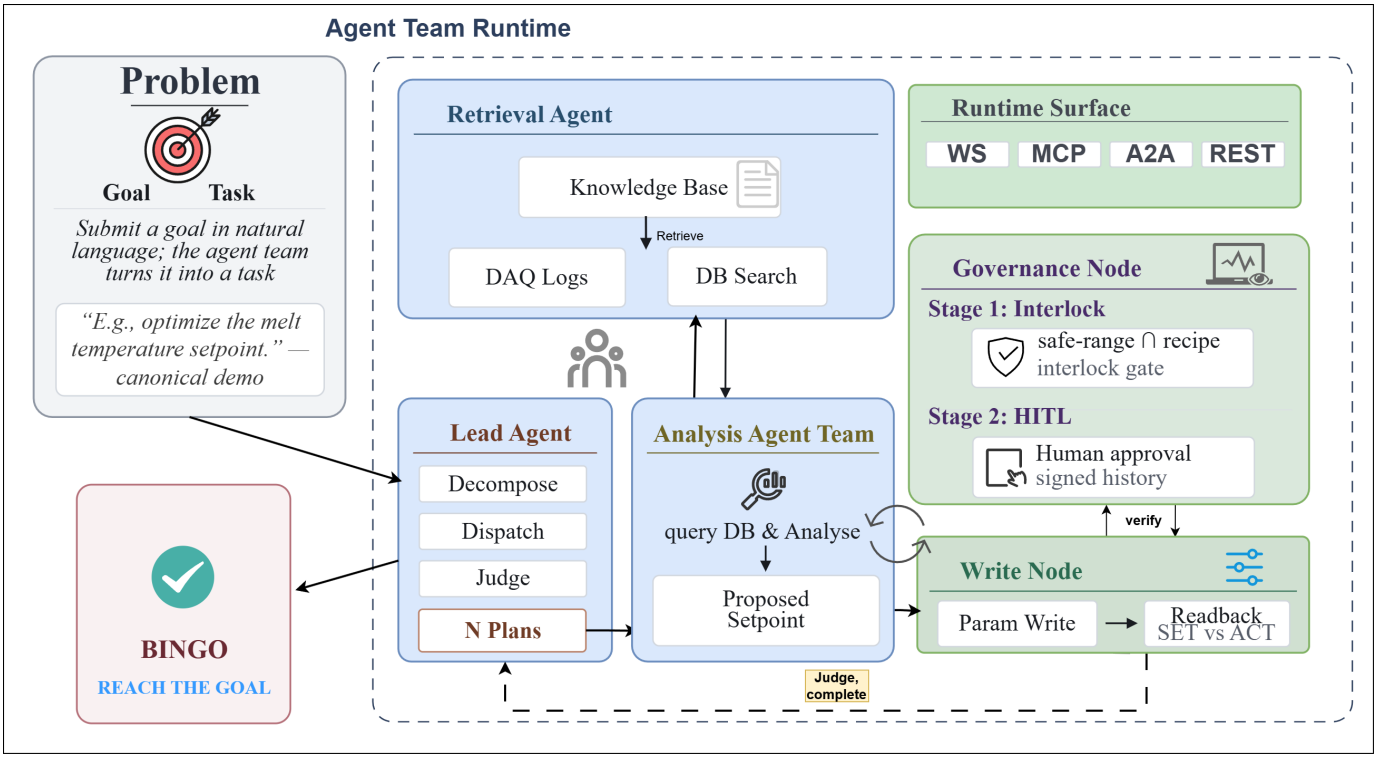


Fig. 3. System-level runtime schematic linking task formulation, agent-team roles, runtime interfaces, governance, and parameter writing. It is an architectural overview, not a strict execution sequence: the approval-gate panel denotes a route-level gate used only on configured routes, and decisions in this study came from a scripted fixture. The *BINGO/REACH THE GOAL* panel denotes the application objective, not an experimental result or a guarantee of autonomous convergence. Section IV describes the code-grounded optimization path: one bounded `optimization_explore` invocation, pre-dispatch checks, route-dependent approval, post-approval revalidation, shared write guards, and separately recorded command and process evidence.

Authorization and value admission are separate checks. The target band describes a desired process outcome; it does not define the admissible write interval. A candidate may satisfy all write constraints while the measured process value remains outside the target band.

For an online node write, the admissible value interval is

$$\mathcal{I}_n^{\text{online}}(t) = \mathcal{H}_n \cap \mathcal{L}_n^{\text{base}} \cap \mathcal{L}_n^{\text{prod}}(t) \cap \mathcal{L}_n^{\text{rec}}(t), \tag{3}$$

where $\mathcal{H}_n$ is the hard engineering range, and $\mathcal{L}_n^{\text{base}}$, $\mathcal{L}_n^{\text{prod}}$, and $\mathcal{L}_n^{\text{rec}}$ are the baseline, active-product, and active-recipe limits. These numerical bounds are distinct from the scope, policy, grant, and binding predicates in Eq. (1). Missing optional limits impose no restriction. An empty intersection rejects the request and identifies the excluding layer. Recipe dispatch checks a replacement recipe against the hard, baseline, and product limits rather than the previous recipe's window. Step, interval, and hold-window limits apply when configured. These application-level checks complement, but do not replace, equipment interlocks.

B. Approval, Execution, and Evidence

Approval behavior depends on the operation route. For a single-node `optimization_explore` request in manual mode, the handler sends the node, requested value, current effective interval, and stated rationale to the approval service, then waits before calling the governed write operation. A decline or timeout returns without reaching the controller write. While the request is pending, the handler does not dispatch. After approval, it rechecks the applicable node binding and channel grant. The shared write controller then evaluates the current engineering, parameter, product, and recipe limits, along with the step and write-interval rules, before dispatching to the configured driver. This single-node path does not use the recipe-operation context snapshot described below.

Recipe operations use a separate approval path. For `recipe_trial` and `recipe_apply`, the approval is bound to the recipe operation and active line–product–recipe–run context. After approval, the handler checks the authority and context again and verifies the recipe definition and candidate scope before dispatch. A changed context, revoked binding, or changed recipe definition prevents that proposal from proceeding. Recipe trial approvals use a 30-minute window by default; single-node approval waits expire after 180 s by default. Online agent and manual node writes are separated by at least 60 s outside benchmark mode, while recipe trials use a per-line cadence of 300 s by default.

Fig. 4 places the admissible interval in Eq. (3) alongside the route-dependent approval and shared-dispatch paths.

A `recipe_trial` illustrates the recipe-operation path. The handler checks the bound recipe, active run, candidate parameters, and route timing. If approval is required, it requests approval and then revalidates authority and context before submitting a bounded batch through the configured driver. It returns a receipt for each parameter. A trial does not save or apply a recipe version; the caller can retrieve later DAQ observations separately before updating, applying, or rolling back the recipe. The driver registry maps semantic controls

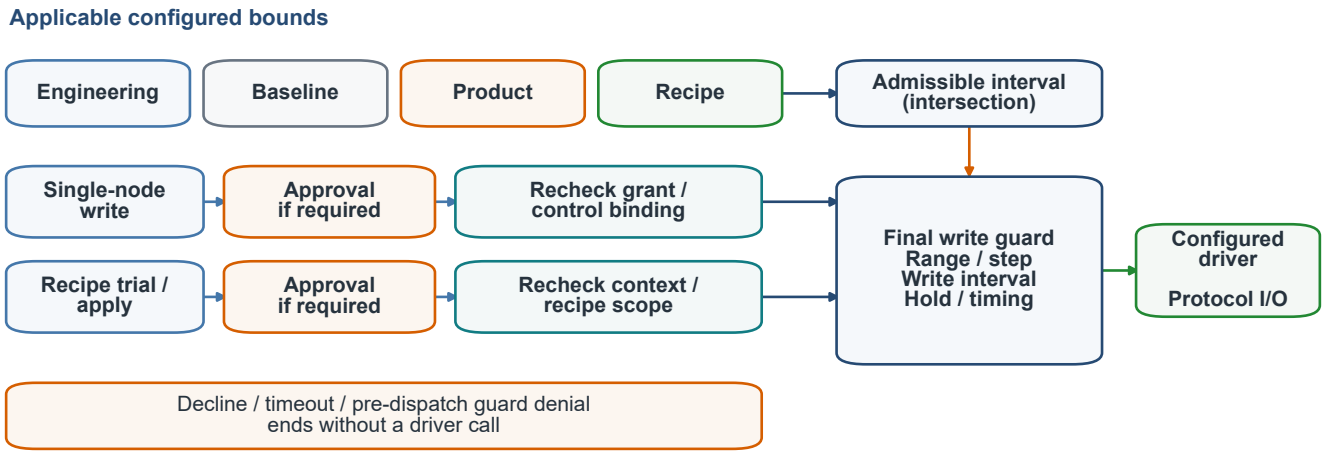


Fig. 4. Configured hard, baseline, product, and recipe bounds are intersected to define the admissible interval; the diagram is categorical and does not encode measured values. Approval is route-dependent: node writes recheck the binding and grant, while recipe operations recheck authority, active context, and recipe scope. Approved and ungated requests pass through the shared range, step, timing, and hold guard before driver dispatch. A decline, timeout, or pre-dispatch denial ends without a driver call.

to configured protocol drivers, so operation arguments do not contain protocol-specific details.

Receipts record what happened to a command; observations record what followed in the process. Command results are graded as readback-verified, transport-acknowledged, or unverified. A readback mismatch can trigger up to three reads at 2 s intervals, and recipe batches retain verification outcomes for each parameter. A readback-verified result means the returned value matched the request. A transport acknowledgement alone does not establish that the device applied the value, and an unverified result does not establish a matching readback. DAQ stores later timestamped process observations, while recipe history, approvals, and task records remain with their owning services. Identifiers and timestamps can link these records when available, but a later observation alone does not prove that the command caused the measured change. Section VI evaluates selected parts of this evidence chain under the stated test conditions.

VI. Experimental Evaluation

The evaluation addresses four questions. RQ1: do configured node–driver mappings accept admissible values, reject structural violations, and return matching readbacks? RQ2: does a governed write path separate a no-write condition from a scripted mission that reaches the target band? RQ3: does the process-window guard block targeted excursions before the driver? RQ4: do approval and authorization routes reach their expected branches? A separate block reports a live-Agent pilot as a negative result. Each block retains its own unit and denominator; counts are not pooled across blocks.

A. Experimental Design

Table II summarizes the six blocks. The design uses a negative control wherever a positive claim is made: the no-write condition (C0) writes nothing, the guard ablation removes one guard at a time, and the approval pilot includes a rejection branch. E1 is a census of the configured writable nodes rather than a sample. E2 pairs conditions by seed and scene but does not time-match them. E4 repeats each arm three times. E5 is a feasibility pilot, not a performance comparison.

B. Execution Protocol

All blocks ran against the same recorded platform snapshot: the run manifests identify source commit `da627d21`, Node 24.19.0, Windows 11, an isolated service on port 3005, and a scenario simulator on port 4011. The experiment runners were uncommitted at execution time and were later committed as `40533741`; the recorded commit therefore identifies the platform source, not a hermetic tree. The probe manifest records configuration hash `a66769851fa03de5`; the ablation manifest records `3a57e722d4764b98`. Two platform lifecycles were used. E1, E2, E4, and the approval and authorization calls ran with `AW_BENCH_MODE=1`, which relaxes normal timing and hold-window guards. The production-guard checks ran in a separate process with the variable absent, and the runner asserted its absence before issuing calls. The two lifecycles were never combined in one result block; consequently, E1, E2, and E4 do not exercise the 60-s interval or hold-window guards.

The run order was: two integrated pipeline executions; E1 probes; E2 C0 and C1 for seeds 42–46; E4 four arms with three repetitions; production-guard branches; approval and authorization calls; and the E5 pilot. The C0 and C1 conditions used the same seed list. C0 held for 60 s without writes; C1 ran the scripted mission through the shared governed-write service and queried DAQ after configured settling intervals. Each C1 seed was executed once and no seed was discarded. The two pipeline executions used the same command and seed, and the comparison tool reported identical decision fields.

Outcomes were taken from independent data planes: HTTP status and response bodies for probes, the platform write ledger for command outcomes, actuator readbacks for control

TABLE II
EXPERIMENTAL DESIGN. BLOCKS DIFFER IN UNIT, DENOMINATOR, AND FIXTURE, SO OUTCOMES ARE REPORTED SEPARATELY.

Block	Question	Unit and repetitions	Fixture	Outcome	Status
E0 Pipeline	Do configured service paths execute end to end?	Assertion; 2 runs × 77	Running platform and simulator	Pass, warning, failure	Pass, 77/77 each
E1 Mapping	RQ1: accept, reject, and read back?	Node request; 55 nodes, 439 requests	Four simulator scenes	Accepted, rejected, matched	Consistency check
E2 Control	RQ2: no-write versus scripted?	Seed–scene pair; 15 pairs	Three simulator scenes	Endpoint in target band	Descriptive demonstration
E3 Routes	RQ4: expected approval and authorization branches?	Route call; 7 calls	Injection scene; scripted decisions	Expected branch and code	Path checks pass
E4 Ablation	RQ3: which guard blocks the excursion?	Arm × repetition; 4 × 3, 12 per repetition	Mock driver, benchmark mode	Violations reaching driver; false blocks	Descriptive, four arms
E5 Agent	Can a live Agent complete a governed write?	Episode; 4 attempts, 2 recorded	omp Agent, injection scene	Writes, band, terminal state	Negative; no claim

values, and DAQ samples for process observations. Agent transcripts and terminal logs were not used as outcome evidence. For E2 we recomputed band membership from the raw `pv`, `target`, and `tol` columns because the runner's `attained` field combines different criteria across conditions. The runner commands, seeds, run identifiers, and raw CSVs are preserved in Supplementary File S7.

Five deviations are reported rather than smoothed over. First, one biaxial-line structural case was not collected, so the structural denominator is 219 rather than 220. Second, HTTP endpoints expose no actuator readback, so 216 of 220 accepted requests have a readback value. Third, the auto-mode approval calls used the same starting values as the manual calls but crossed runner labels, so the four calls are not a matched approval-on/off design. Fourth, the cast-film probe record was excluded: its manifest declared four scenes but recorded only cast-film, and its simulator endpoint did not match the reproduction command. Its 24 admissible requests, 24 structural probes, and 20 readbacks are not pooled. The film-line preset check (nine selected negative probes rejected without code changes) is a configuration check, not matrix coverage. Fifth, one C0 control began from residual state; this is reported with the target-band results.

C. Integration and Mapping Results

Two integrated executions each returned 77 passing assertions, zero warnings, and zero failures across 17 phases. The unit is an assertion within a run, not an independent trial, so the two runs support reproducibility of the configured path but not a success rate.

Table III reports the collected scene–protocol subset. The 55 configured writable nodes span five configured protocol variants across four families: Modbus over TCP and RTU, OPC UA, MQTT, and HTTP. Each scene used four of the five variants. Each node received three interior values, the tested upper boundary, and four structural-range requests. All 220 admissible requests were accepted, all 219 collected structural probes were rejected, and all 216 available readbacks matched. The admissible and structural values are computed from the same configured minimum and maximum used by the guard, so this is a consistency check of the configured mapping, not an independent estimate of range correctness. In the simulator, a readback is an echo of the configured value; it demonstrates transport and register mapping, not actuator application. Because the probes are a census of the configured nodes, clustered at node level, we report counts rather than a population-level interval.

The biaxial line contributes 120 admissible requests, 119 structural probes, and 116 readbacks; its HTTP endpoint has no readback, and one structural case was not collected. The film-line preset check reused the running platform, so it is not an independent cold-start deployment. The selected mappings therefore passed their configured checks, but the full scene–protocol matrix remains incomplete.

D. Target-Band Comparison

C0 and C1 used seeds 42–46 in the injection, wastewater, and annealing scenes. C0 made no writes for 60 s; its endpoint is the mean of the final five process samples. C1 ran the deterministic scripted mission through the `recipe_trial` batch route, not the single-node `optimization_explore` route described in Section IV, and queried DAQ after configured settling intervals. The mission stops at the first in-band reading, so its endpoint is the stopping value, not a fixed-horizon measurement. C1 took 150.41–172.90 s per seed, so the conditions differ in exposure time and mission logic. C1 is a positive control, not a competing industrial controller; the experiment contains no PID or model-predictive baseline.

For injection mass, the target was 32.5 ± 0.35 g (32.15–32.85 g). C0 endpoints were 31.206–31.272 g (mean 31.244 g), while C1 endpoints were 32.173–32.340 g (mean 32.241 g). For wastewater dissolved oxygen (DO), the target was 3.4 ± 0.6 mg/L (2.8–4.0 mg/L). C0 endpoints were 0.198–0.268 mg/L (mean 0.219 mg/L), while C1 endpoints were 3.576–3.864 mg/L (mean 3.714 mg/L). For annealing, the target was 95 ± 6 HV (89–101 HV). C0 endpoints were 134.36–134.90 HV (mean 134.64 HV), while C1 endpoints were 97.44–99.90 HV (mean 98.22 HV).

One C0 control began from residual state: seed 42 started at 32.378 g injection mass and 2.894 mg/L DO, both inside their bands, rather than at the blueprint state used by C1. Seeds 43–46 began from the blueprint. Across the 12 blueprint-matched pairs, 0/12 C0 and 12/12 C1 endpoints were in band; including the warm-started seed-42 control gives 0/15 and 15/15. We report counts rather than a p-value: the conditions are not time-matched, the C1 endpoint is the stopping value,

TABLE III
OBSERVED SCENE–PROTOCOL PROBE OUTCOMES. THE TABLE REPORTS THE COLLECTED SUBSET; READBACK DENOMINATORS INCLUDE ONLY ACCEPTED REQUESTS WITH AN AVAILABLE ACTUATOR VALUE.

Scene	Writable nodes	Writable nodes by protocol	Admissible accepted	Structural rejected	Readback matched
Injection	11	TCP 5; RTU 2; OPC UA 3; MQTT 1	44/44	44/44	44/44
Wastewater	7	TCP 1; RTU 3; OPC UA 2; MQTT 1	28/28	28/28	28/28
Annealing	7	TCP 3; RTU 2; OPC UA 1; MQTT 1	28/28	28/28	28/28
Biaxial line	30	TCP 20; RTU 1; OPC UA 8; HTTP 1	120/120	119/119	116/116

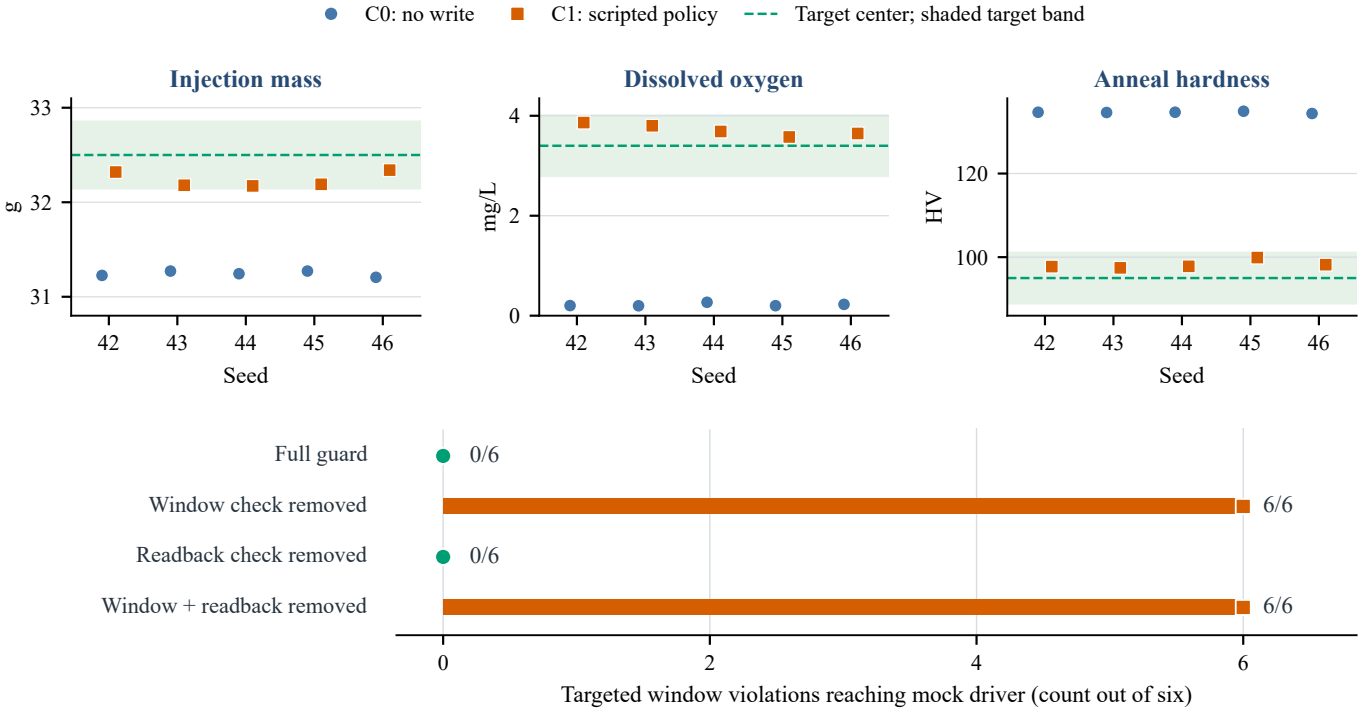


Fig. 5. Scripted-policy endpoints and process-window guard ablation results. Top: terminal process-variable statistics for seeds 42–46 in the injection, wastewater dissolved-oxygen, and annealing scenes. C0 is the 60-s no-write condition; each value is the mean of the final five samples, and the seed-42 control began from residual state. C1 is the deterministic scripted policy; each value is the first in-band reading at which the mission stopped. Shaded regions and dashed lines mark the target bands and their centers. Bottom: targeted process-window violations reaching the mock driver under each guard configuration (six probes per arm, three repetitions). The process-window check was removed in the second and fourth arms; the readback check was removed in the third and fourth. Structural checks remained active in all arms.

and the 15 pairs are clustered within three scenes. A paired test would overstate the design. Across the five C1 seeds, the missions issued 12 writes in 17 rounds for injection, 56 in 59 for wastewater, and 41 in 50 for annealing; these are workload counts, not an efficiency comparison. Fig. 5 shows each endpoint and its band.

E. Process-Window Guard Ablation

With the full guard, 18/18 negative probes were rejected before the mock driver and 0/6 targeted window violations reached it. Removing only the process-window guard reduced negative-probe rejections to 12/18: all six targeted violations reached the driver, while the 12 structural probes remained blocked. The no-readback arm retained the process-window guard and produced the same 18/18 rejection result. The combined no-window/no-readback arm matched the `no-interlock` arm on these stimuli. Each arm also accepted 9/9 nominal legal requests without a false block and met its configured boundary expectations in 9/9 checks. These are three technical repetitions of deterministic stimuli, not independent trials; the inferential unit would be the stimulus type (six negative and two targeted), not the 18 submissions. We therefore report counts and do not attach p-values.

No readback-mismatch stimulus was included, so the experiment isolates the process-window predicate but says nothing about readback-check effectiveness or interaction between the two checks. The ablation ran in benchmark mode against a mock driver and therefore assesses the software guard in that fixture, not plant-level protection. In a separate production-mode run, the legal baseline value 190 was accepted and one example each was rejected for interval (429, `WRITE_INTERVAL_NOT_ELAPSED`), step (409, `STEP_LIMIT_EXCEEDED`), node range (400, `NODE_RANGE_EXCEEDED`), recipe window (400, `RECIPE_LIMIT_EXCEEDED`), and hold window (429, `WRITE_FREQUENT`). These single-case checks confirm expected branches; they are not rejection-rate estimates.

F. Approval and Authorization Path Checks

A four-call simulator pilot exercised the approval route. In manual mode, the approved case changed the control value from 202 to 204.8, while the rejected case left it at 216. The two auto-mode calls began at 202 and 216, the same starting values as the manual calls, but their runner labels were crossed relative to the manual decisions. The decisions came from a test fixture, no operator made them, and the calls do not form a matched approval-on/off comparison. We therefore treat these records as a path check rather than an estimate of the approval effect. A separate three-call authorization probe rejected one unbound write and one cross-node write while accepting one write to a bound control node (240 to 241). These examples show that the configured checks returned their expected outcomes in the tested routes; they do not establish a general safety rate or live-Agent decision quality.

G. Agent-Route Pilot and Negative Result

The live-Agent orchestration path was exercised in a separate pilot with an omp lead–worker team on the injection scene; no governed write reached the dispatcher. Four setup attempts were made, and two produced recorded episodes. Both recorded episodes issued zero governed writes. The first ended at 301.48 s with terminal state `COMPLETED` while the process variable was 31.28 g, outside the 32.15–32.85 g band; this exposed a platform defect in which goal completion was not checked against the goal criterion. The second ended at 902.84 s with `BUDGET_TIMEOUT` and a process variable of 31.23 g. The two earlier attempts did not produce result rows: one task remained queued and one worker lacked the host-tool mount. The pilot also exposed task-artifact addressing and lead-queue draining defects, which were bypassed in the runner.

We report this pilot because omitting it would overstate the evidence; it is not used for any performance claim. It shows that the deterministic blocks characterize the runtime path, not an autonomous Agent policy. A confirmatory study requires the platform fixes in Section VII and a larger episode budget.

H. Scope of Inference

Four limits bound the claims. Construct: band membership is a task-level endpoint, not a control-performance metric; settling time, overshoot, and stability were not measured. Internal: C0 and C1 differ in duration and mission logic, the C1 endpoint is the stopping value, the ablation isolates one predicate against a mock driver, and the approval decisions were scripted. External: the evidence covers four simulator scenes, one mock driver, and no physical plant, cold-start deployment, or competing controller. Statistical: the study reports descriptive counts; because stimuli are clustered in nodes and scenes and the conditions are not time-matched, we do not attach population-level intervals or p-values. These limits are reflected in the claims and in the conclusion.

VII. Discussion

The probes and missions support different claims. The probes show that selected configurations resolved to the intended node and driver paths; the missions show that a deterministic policy issued governed writes and returned endpoints in band in three simulator scenes. Neither block isolates the write-path effect: the conditions differed in duration and mission logic, and C1 stopped at its first in-band reading. The planned scene–protocol matrix remains incomplete, and the film-line check reused a running platform.

The ablation supports a narrower causal claim: removing the process-window guard allowed all six targeted excursions to reach the mock driver while structural checks remained active. Because it ran in benchmark mode with a mock driver and no readback-mismatch stimulus, it does not assess the full production guard stack. Production-mode examples exercised the interval, step, node-range, recipe-window, and hold-window branches, but each is a route check rather than a rate estimate. Approval decisions were scripted and not matched, so the pilot cannot establish an operator-decision effect. The runtime exposes a common operation contract to registered harness adapters, but no active harness was used in the deterministic blocks. The Agent pilot issued no governed writes and is reported as a negative result.

The first recorded Agent episode is more consequential than its zero-write outcome: the platform marked the task `COMPLETED` while the process variable was outside the target band. A runtime that treats task completion as evidence of goal satisfaction would hide a failed operation. Goal completion should therefore be a postcondition checked against the same independent data plane used for evaluation, not a status reported by the orchestration layer. This is a design requirement for the harness path, not a property demonstrated by the present experiments.

The evidence model also constrains deployment evaluation. A command receipt, an actuator readback, and a DAQ sample record different things; the tests do not verify that every configured observation is the commissioned response to the selected control point. Before using measurements to assess a write, each scene needs a verified control–measurement pairing and observation delay.

The manifests identify commit `da627d21` and show that the platform and simulator were already running; they do not capture a hermetic snapshot of all active service source. Supplementary File S7 retains the runner inputs and records needed to audit the reported runs. Application-level checks supplement PLC, SIS, and equipment protection; the simulator and mock-driver tests do not establish physical-plant safety. Time-matched controls, a PID or rule baseline, cold-start deployment, and a positive live-harness write remain future work.

VIII. Conclusion

This work presents a configuration-driven runtime that binds production context and named signals to operations, registered protocol drivers, and shared write governance. Across the collected four-scene simulator probes, 220/220 admissible requests were accepted, 219/219 collected structural-range requests were rejected, and 216/216 available actuator readbacks matched. In the three-scene scripted comparison, 15/15 C1

runs ended in band, while the 60-s no-write condition did not; 12 pairs began from a matched blueprint state, and one C0 control was warm-started. The comparison is descriptive because exposure times, mission logic, and endpoints differed. In the mock-driver ablation, the process-window guard blocked all six targeted violations, whereas removing it forwarded all six while structural checks remained active. A separate Agent-route pilot issued no governed writes; one recorded episode was marked `COMPLETED` while the process variable was outside the band, which motivates a completion postcondition. In these simulation and mock-driver settings, the results demonstrate the selected scene mappings, scripted-write path, and process-window guard. They do not establish live-Agent or harness behavior, human-approval effects, or physical-plant protection.

ACKNOWLEDGMENT

The authors used AI-assisted tools for language editing, code drafting, and figure preparation, and take responsibility for the manuscript. Identities and funding are withheld in this review copy.

AUTHOR CONTRIBUTIONS

Individual CRediT contributions are declared in the submission system and withheld in this review copy.

DECLARATION OF COMPETING INTEREST

The authors declare no known competing financial interests or personal relationships that could have appeared to influence this work.

DATA AVAILABILITY

Supplementary File S7 contains the run manifests, raw CSVs, per-seed outcomes, ablation records, and a visual report. The reported commit and harness hashes are provenance fields; the package contains no physical-plant data.

REFERENCES

[1] A. Biskupovic, A. Villalonga, F. Castaño, R. E. Haber, and F. Núñez, "Agent-based supervision for service-oriented industrial cyber–physical systems," *IEEE Trans. Ind. Inform.*, vol. 21, no. 3, pp. 2719–2728, 2025, doi: 10.1109/TII.2024.3514178.

[2] Y. Xia, N. Jazdi, J. Zhang, C. Shah, and M. Weyrich, "Control industrial automation system with large language model agents," in *Proc. IEEE 30th Int. Conf. on Emerging Technologies and Factory Automation (ETFA)*, 2025, pp. 1–8, doi: 10.1109/ETFA65518.2025.11205539.

[3] K.-C. Huang, H.-T. Lin, C.-H. Lin, and M.-T. Huang, "Large language model integration in industrial cyber-physical systems: A multi-modal architecture for automated control generation," in *Proc. Int. Conf. on Control, Automation and Diagnosis (ICCAD)*, 2026, pp. 1–6, doi: 10.1109/ICCAD69956.2026.11643124.

[4] K. Köhle, D. Anicic, T. A. Runkler, and R. Graf, "LLM-driven multi-agent control for skill-based smart manufacturing," in *Proc. IEEE 31st Int. Conf. on Emerging Technologies and Factory Automation (ETFA)*, 2026, pp. 1–8, doi: 10.1109/ETFA68377.2026.11725257.

[5] Z. Liu *et al.*, "Agents4PLC: Automating closed-loop PLC code generation and verification in industrial control systems using LLM-based agents," *IEEE Trans. Softw. Eng.*, vol. 52, no. 5, pp. 1672–1687, 2026, doi: 10.1109/TSE.2026.3667895.

[6] R. Wu, J. Ai, and T. Li, "InstructMPC: A human-LLM-in-the-loop framework for context-aware control," in *Proc. IEEE 64th Conf. on Decision and Control (CDC)*, 2025, pp. 172–179, doi: 10.1109/CDC57313.2025.11312156.

[7] J. Lim and I. Kovalenko, "A large language model-enabled control architecture for dynamic resource capability exploration in multi-agent manufacturing systems," in *Proc. IEEE 21st Int. Conf. on Automation Science and Engineering (CASE)*, 2025, pp. 2088–2095, doi: 10.1109/CASE58245.2025.11163802.

[8] Z. Zhao *et al.*, "A large language model-based multi-agent manufacturing system for intelligent shopfloors," *Adv. Eng. Inform.*, vol. 69, 2026, art. no. 103888; doi: 10.1016/j.aei.2025.103888.

[9] B. Hofmann, N. Piechulek, S. Kreitlein, J. Franke, and P. Bruendl, "Beyond touch-based human-machine interface: Control your machines in natural language by utilizing large language models and OPC UA," *arXiv preprint arXiv:2510.11300*, 2025. [Online]. Available: https://arxiv.org/abs/2510.11300v2

[10] Y. Liu, X. Xu, and J. Polzer, "Agentic AI-enabled semantic commissioning of a cognitive digital twin for reconfigurable manufacturing," in *Proc. IEEE 22nd Int. Conf. on Automation Science and Engineering (CASE)*, 2026, doi: 10.1109/CASE69030.2026.11704436.

[11] L. M. Vieira da Silva, A. Köcher, and F. Gehlhoff, "Beyond formal semantics for capabilities and skills: Model context protocol in manufacturing," in *Proc. IEEE 30th Int. Conf. on Emerging Technologies and Factory Automation (ETFA)*, 2025, pp. 1–4, doi: 10.1109/ETFA65518.2025.11205601.

[12] K. Köhle, D. Anicic, T. A. Runkler, K. Dorofeev, A. S. Thuluva, and R. Graf, "Virtually commissioned semantic recipes based on OPC UA skills," in *Proc. IEEE 8th Int. Conf. on Industrial Cyber-Physical Systems (ICPS)*, 2025, pp. 1–6, doi: 10.1109/ICPS65515.2025.11087832.

[13] M. Xavier, M. Jolly, V. M A, and M. Xavier, "IndustriConnect: MCP adapters and mock-first evaluation for AI-assisted industrial operations," 2026, arXiv:2603.24703 [cs.SE]. [Online]. Available: https://arxiv.org/abs/2603.24703

[14] C. Graul, W. Żarski, D. Mikołajewski, and I. Rojek, "Ai agents in industry 4.0: AAS–OPC UA–LLM architecture as the foundation of intelligent manufacturing systems in the context of industrial enterprise implementation," *Applied Sciences*, vol. 16, no. 11, p. 5428, 2026, doi: 10.3390/app16115428.

[15] D. Schall, "Safe integration of large language models into industrial process control: A multi-agent architecture with P&ID-grounded validation," *Autonomous Intelligent Systems*, vol. 6, 2026, article no. 14; doi: 10.1007/s43684-026-00136-1.

[16] S.-C. Lin and C.-Y. Chen, "An integrated framework featuring policy-governed agentic AI for closed-loop manufacturing control with multi-source sensor–MES–ERP," *The International Journal of Advanced Manufacturing Technology*, vol. 143, pp. 6405–6428, 2026, doi: 10.1007/s00170-026-17806-2.

[17] Y. Xia, N. Jazdi, and M. Weyrich, "An architecture for integrating large language models with digital twins and automation systems," in *Proc. IEEE 30th Int. Conf. on Emerging Technologies and Factory Automation (ETFA)*, 2025, pp. 1–8, doi: 10.1109/ETFA65518.2025.11205636.

[18] H. Wang, C. M. Poskitt, and J. Sun, "AgentSpec: Customizable runtime enforcement for safe and reliable LLM agents," in *Proc. IEEE/ACM 48th Int. Conf. on Software Engineering (ICSE)*, 2026, pp. 2938–2950, doi: 10.1145/3744916.3764546.

[19] E. Debenedetti *et al.*, "Defeating prompt injections by design," 2025, arXiv preprint arXiv:2503.18813. [Online]. Available: https://arxiv.org/abs/2503.18813

[20] IEC, "IEC 62443-3-3:2013: Industrial communication networks—network and system security—part 3-3: System security requirements and security levels," Standard, 2013.

[21] ——, "IEC 61511-1:2016: Functional safety—safety instrumented systems for the process industry sector," International Standard, 2016.

[22] Model Context Protocol contributors, "Model Context Protocol: Tools," Specification, 2026, accessed: Oct. 20, 2026. [Online]. Available: https://modelcontextprotocol.io/specification/2026-07-28/server/tools

[23] ISA, "ISA-88.00.01-2010: Batch control—part 1: Models and terminology," International Standard, 2010.

[24] ——, "ANSI/ISA-95.00.01-2025 (IEC 62264-1 mod): Enterprise-control system integration—part 1: Models and terminology," International Standard, 2025.

[25] J. Moyne, E. del Castillo, and A. M. Hurwitz, Eds., *Run-to-Run Control in Semiconductor Manufacturing*, 1st ed. Boca Raton, FL: CRC Press, 2000.

[26] R. Parasuraman, T. B. Sheridan, and C. D. Wickens, "A model for types and levels of human interaction with automation," *IEEE Trans. Syst., Man, Cybern., Part A*, vol. 30, no. 3, pp. 286–297, 2000, doi: 10.1109/3468.844354.